Project Management

Identify each group member's responsibilities.

For our Numerical Root Finder project, we divided the tasks among our group members to make the work more organized and manageable. Each member is assigned a specific part of the project so we can complete the requirements and keep track of our progress.

1. Natividad, Aaron Matthew: Project Understanding and Project Management: Responsible for explaining the project's purpose, features, inputs, outputs, and limitations. Also handles task distribution, project scheduling, and identifying possible challenges.

2. Lim, Kyra Michiko: Algorithm Design (Items 1–2): Responsible for identifying and explaining the Bisection Method and preparing a flowchart or pseudocode showing how the algorithm works.

3. Co, Reine Nathaniel: Program Decomposition (Items 3–5): Responsible for identifying the main program components, proposing the necessary Python functions, and explaining how these functions will work together.

4. Fernandez. Akima Lou: Development Environment and Technical Preparation: Responsible for identifying the software, libraries, and tools needed for development. Also prepares the proposed GUI layout and application structure.

5. Garcia, Nathan Drake: GitHub Repository Initialization: Responsible for setting up the GitHub repository, adding collaborators, organizing the project files, and preparing the README and Git workflow.

All members will also help review, test, and improve the application during development.

Prepare an initial task assignment matrix.

The following table shows the assigned tasks and expected outputs for each group member. We divided the responsibilities based on the different parts of our Numerical Root Finder project to make sure everyone has a specific task to accomplish.

Group Member	Assigned Task	Responsibilities	Expected Output
Natividad, Aaron Matthew	Parts A and E – Project Understanding and Management	Explain the project, identify its requirements, manage task assignments, and prepare the development timeline	Project proposal, requirements analysis, task matrix, and development schedule
Lim, Kyra Michiko	Part B1–B2 – Algorithm Design	Explain the Bisection Method and create the algorithm flowchart	Algorithm explanation and flowchart

Co, Reine Nathaniel	Part B3–B5 – Program Decomposition	Identify program components, propose Python functions, and explain their interactions	Functional decomposition and proposed Python functions
Fernandez, Akima Lou	Part C – Technical Preparation	Identify development tools, libraries, formulas, and prepare the GUI wireframe	Development environment setup, project structure, and GUI design
Garcia, Nathan Drake	Part D – GitHub Initialization	Create the GitHub repository, add collaborators, organize files, and prepare repository documentation	GitHub repository, README, initial commits, and supporting evidence

All members will also participate in developing, testing, reviewing, and presenting the final application. We will coordinate our work to make sure the different parts of the project are completed and properly connected.

Develop a schedule covering the remaining progress reports and final submission.

To manage our project properly, we prepared an initial development timeline. We divided the work into different stages so we can monitor our progress and complete the requirements before the final submission.

Development Stage	Planned Activities	Expected Output
Stage 1 – Planning and Preparation	Complete the project proposal, requirements analysis, algorithm flowchart, initial GUI wireframe, task assignments, and GitHub setup	Complete planning documents and Progress Report 1
Stage 2 – Core Program Development	Develop the polynomial evaluation function, input validation, and Bisection Method algorithm	Working root-finding algorithm and development progress report
Stage 3 – GUI Development and Integration	Create the user interface, connect the algorithm, add the iteration table, and display the polynomial graph	Initial working desktop application
Stage 4 – Testing and Improvements	Test the program using different cubic equations, check invalid inputs, and fix errors	Tested application, recorded test results, and progress report
Stage 5 – Final Documentation and Submission	Complete the README, gather screenshots, finalize the Python files, and prepare the final presentation	Complete final project requirements and presentation

This schedule will serve as our guide throughout the development of the project. We will adjust the timeline based on the actual deadlines provided by our professor. We also plan to regularly check each member's progress to make sure our tasks are completed on time.

Identify possible technical challenges and proposed solutions.

During the development of our Numerical Root Finder, we may encounter technical challenges that can affect the accuracy and functionality of the application. We identified the following possible problems and proposed solutions to help us prepare.

1. Incorrect Root Calculations

One possible problem is getting inaccurate results due to errors in the Bisection Method calculations or interval updates.

Proposed Solution: We will test our algorithm using cubic equations with known roots and compare the calculated results to check if our program is working correctly.

2. Invalid User Inputs

Users may enter letters instead of numbers, incorrect coefficients, negative tolerance values, or intervals that do not meet the requirements.

Proposed Solution: We will implement input validation and display clear error messages whenever the user enters invalid values. This will help prevent the program from crashing.

3. Graphing and User Interface Problems

We may encounter difficulties displaying the cubic graph, updating the iteration table, or arranging the interface components properly.

Proposed Solution: We will test each GUI component separately before combining everything into the application. We will also make sure the interface is simple and easy to understand.

4. Integration of Python Functions

Since our application will be divided into different functions and files, some components may not work correctly when connected.

Proposed Solution: We will organize our code into separate modules, use consistent function names, and test how the components interact with one another.

5. GitHub Collaboration Issues

Working on the same repository may cause problems such as merge conflicts, missing files, or accidentally overwriting another member's changes.

Proposed Solution: We will regularly update the repository, coordinate our changes, and use separate branches when needed to avoid conflicts.

By identifying these possible challenges early, our group can prepare solutions and reduce errors during development. This will also help us complete the project more efficiently and make sure our application meets the required specifications.